

How much steel is in this bicycle?

Programming and Tools for AI – Week 3

James McDermott



**Part 1: A bicycle is made of
parts**

A bill of materials (BOM)

A factory describes a bicycle as a **bill of materials**, ie parts made of parts:

bicycle

wheel	x2		
rim	x1	aluminium	380 g
spoke	x32	steel	5 g
hub	x1	steel	200 g
tyre	x1	rubber	320 g
frame	x1	steel	2200 g
handlebar	x1	aluminium	300 g
saddle	x1	plastic	260 g
chain	x1	steel	300 g

The questions it has to answer

- What does the whole bicycle weigh?
- **How much steel is in it?** (The scrap merchant wants to know.)
- What does one wheel cost, with everything in it?
- How many parts of each material?
- What is the single heaviest part?

We will write **one** function to traverse the tree, to answer all of them.

First, choose a representation

Before writing any code, we need to know what the data will look like.

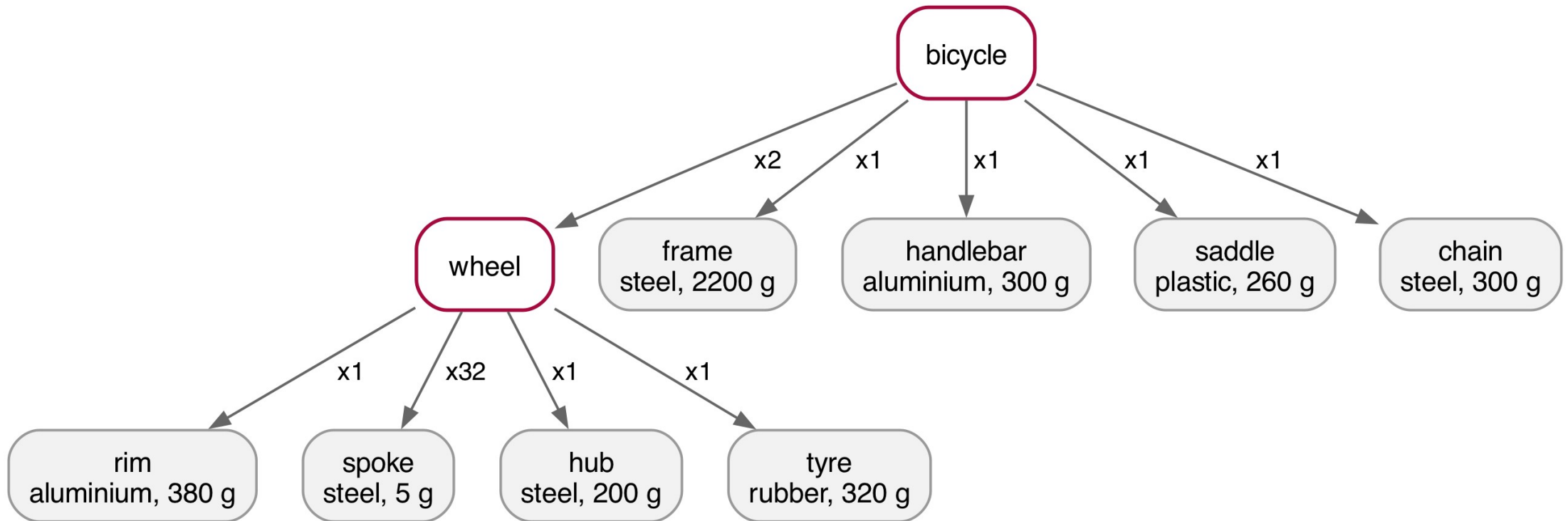
The bicycle is a **tree**: it is made of parts which are made of parts, etc, until we reach basic parts.

Two possible encodings:

- **Nested** — each part holds its own children.
- **Flat** — one big table of name -> children, looked up by name.

We will use **nested**, as it makes our code easier for now.

The bicycle, as a tree



The bicycle, as a dict

```
{"name": "bicycle", "qty": 1, "children": [  
  {"name": "wheel", "qty": 2, "children": [  
    {"name": "rim", "qty": 1, "material": "aluminium", "mass": 380, "cost": 24.00},  
    {"name": "spoke", "qty": 32, "material": "steel", "mass": 5, "cost": 0.40},  
    {"name": "hub", "qty": 1, "material": "steel", "mass": 200, "cost": 12.00},  
    {"name": "tyre", "qty": 1, "material": "rubber", "mass": 320, "cost": 18.00}  
  ]},  
  {"name": "frame", "qty": 1, "material": "steel", "mass": 2200, "cost": 95.00},  
  {"name": "handlebar", "qty": 1, "material": "aluminium", "mass": 300, "cost": 22.00},  
  {"name": "saddle", "qty": 1, "material": "plastic", "mass": 260, "cost": 15.00},  
  {"name": "chain", "qty": 1, "material": "steel", "mass": 300, "cost": 14.00}  
]}
```

The bicycle, as a dict

Reminder: dict data structure

```
>>> spoke = {"name": "spoke", "qty": 32, "mass": 5} # create
>>> spoke["mass"] # look up key to get value
5
>>> spoke["material"] = "steel" # add a new key
>>> spoke["qty"] = 36 # overwrite existing key
>>> "cost" in spoke # check if present
False
>>> len(spoke) # number of keys present
4
>>> spoke["colour"]
KeyError
```

Reminder: walking a dict

```
>>> for key in spoke:
...     print(key)
name
qty
mass
material
```

Iterating a dict gives you its **keys**, not its values.

Nested data structures

A dict can store other dicts, a list can store other lists, a list can store dicts, etc.

In our bicycle, children is a **list of dicts**, and each of those might have children of its own, etc.

```
>>> bicycle["children"][0]["name"]  
'wheel'  
>>> bicycle["children"][0]["children"][1]["name"]  
'spoke'
```

Reading left to right: the bicycle's children, the first of them, its children, the second of those. (We don't usually access elements manually like this – it is just a demo.)

Part 2: Walking the bicycle

Code shape matches data shape

We want to calculate the mass of a bicycle.

The data is a part, which contains parts, which contain parts...

So the code is a function, which calls itself, which calls itself...

In other words, it is **recursive**.

Recursion

Recursion is already familiar to you, eg:

```
def factorial(n):  
    if n <= 1:  
        return 1 # base case  
    else:  
        return n * factorial(n-1) # recursive case
```

- The base case **must not** call the function again.
- The recursive case must move **towards** the base case, ie each call gets a smaller input.

Recursive traversal of a tree to calculate total mass

```
def unit_mass(node):  
    if "mass" in node:      # base case: a part we buy  
        return node["mass"]  
    else:                  # recursive case: a part we build  
        total = 0  
        for c in node["children"]:  
            total += c["qty"] * unit_mass(c)  
        return total  
>>> unit_mass(bicycle)  
5180
```

- It's hard to trace the function in your head. Instead focus on the two cases. Does the base case work for leaves? Does the recursive case work for non-leaves?
- Notice again, the recursive case moves towards the base case, this time putting a **smaller part of the tree** into the recursive call, that is the child c.

Common error: forgetting the base case

```
def broken_unit_mass(node):  
    total = 0  
    for c in node["children"]:  
        total += c["qty"] * broken_unit_mass(c)  
    return total
```

KeyError: 'children'

The recursion reached a spoke, which has no children, and there was no base case to catch it.

The same function can be used on a subtree

```
>>> wheel = bicycle["children"][0]
>>> unit_mass(wheel)
1060
```

`unit_mass(node)` is the mass of **one** node, everything inside it included.

A bicycle needs two wheels, but the 2 is applied at the **bicycle** level, in `c["qty"] * unit_mass(c)`. Therefore `unit_mass(wheel)` is correct for one wheel.

Another common error

What went wrong here?

```
def broken_unit_mass(node):  
    if "mass" in node:      # base case: a part we buy  
        return node["mass"]  
    else:                   # recursive case: a part we build  
        total = 0  
        for c in node["children"]:  
            total += broken_unit_mass(c)  
        return total  
>>> unit_mass(bicycle)  
3965
```

We forgot to multiply by `c["qty"]`. We still get an answer, and it might look right. Luckily we have doctests to check!

Accumulator, or comprehension

Here is the recursive case again:

```
total = 0
for c in node["children"]:
    total += c["qty"] * unit_mass(c)
return total
```

This is the **accumulator pattern**, where we start at zero, loop, and add at each step.

Now the same written on one line, using a **comprehension**:

```
return sum(c["qty"] * unit_mass(c) for c in node["children"])
```

A comprehension is “an inside-out for loop”. A useful function `sum` hides the work of starting at zero and adding all the items.

Four types of comprehension

```
>>> L = [1, 2, 3, 4]
>>> Q = [x**2 for x in L] # list comp
[1, 4, 9, 16]
>>> S = {x**2 for x in L} # set comp
{1, 4, 9, 16}
>>> D = {x: x**2 for x in L} # dict comp
{1: 1, 2: 4, 3: 9, 4: 16}
>>> sum(x**2 for x in L) # generator comp
30
```

- A list comprehension creates a new list
- *Mutatis mutandis* for a set comp and for a dict comp
- A generator comprehension creates a generator, and typically we iterate over it immediately.

Sum of steel mass

Suppose a scrap merchant wants to know the sum of mass of steel parts only. Rather than write a second function, we pass in the question:

```
def everything(node):  
    return True # no matter what `node` is, keep it  
  
def unit_mass(node, keep=everything):  
    if "mass" in node:  
        return node["mass"] if keep(node) else 0  
    return sum(c["qty"] * unit_mass(c, keep) for c in  
node["children"])
```

keep is a **function passed as an argument**. We pass the function everything, not the return value everything().

This still sums the total mass as before.

Sum of steel mass

```
>>> def is_steel(node):  
...     return node["material"] == "steel"  
  
>>> unit_mass(bicycle)                # everything  
5180  
>>> unit_mass(bicycle, keep=is_steel) # for scrap merchant  
3220
```

This is nice Lego-like code, because a single function can be used in several ways. Because `keep` has a **default** everything, the calls we had already written still work. We could also sum the cost instead of the mass. We'll see this in the exercises.

Conditional expressions

```
node["mass"] if keep(node) else 0
```

A `if C else B` is an **expression**: it produces a value, so it can sit inside a bigger expression. Different from a normal `if`, which is a statement and produces nothing.

To evaluate a conditional expression, remember the **substitution model**.

```
>>> x = -5
>>> print("negative" if x < 0 else "non-negative")
negative
```

Use it for one small choice. Anything longer belongs in a real `if`.

Useful built-in functions

<code>sum(xs)</code>	total of a sequence of numbers
<code>len(xs)</code>	how many items in a string, list, dict...
<code>min(xs), max(xs)</code>	smallest, largest
<code>sorted(xs)</code>	a new sorted list
<code>any(xs), all(xs)</code>	is any / are all of them true?
<code>enumerate(xs)</code>	pairs each item with its position
<code>zip(xs, ys)</code>	walks two sequences together

Part 3: More questions

Collecting the parts

Some questions are not sums. “Which part is heaviest?” needs the parts themselves, so gather them into a list.

```
def parts(node):  
    if "children" not in node:  
        return [node]  
    out = []  
    for c in node["children"]:  
        out.extend(parts(c))  
    return out
```

Again this is recursive.

As you know append adds one item. extend appends all the items of another list.

What the parts list gives you

```
>>> len(parts(bicycle))  
8
```

This counts one entry per **line in the bill**, not one per physical part.

Counting by type

How many items of each material?

```
>>> from collections import Counter
>>> Counter(p["material"] for p in parts(bicycle))
Counter({'steel': 4, 'aluminium': 2, 'rubber': 1, 'plastic': 1})
```

Again we use Counter, a type of dict that just counts, and a comprehension to create its input.

Collecting by type

What items are made of steel?

```
>>> from collections import defaultdict
>>> g = defaultdict(list)
>>> for p in parts(bicycle):
...     g[p["material"]].append(p["name"])
>>> g["steel"]
['spoke', 'hub', 'frame', 'chain']
```

Again we use `defaultdict`, when we want to collect rather than count. Without `defaultdict`, `g["steel"]` would raise `KeyError` the first time.

sort and sorted

```
>>> xs = [3, 1, 2]
>>> sorted(xs)      # returns a new list
[1, 2, 3]
>>> xs              # the original is untouched
[3, 1, 2]
>>> xs.sort()       # sorts in place, returns None
>>> xs
[1, 2, 3]
>>> xs = xs.sort()
```

sorted works on anything you can iterate. .sort() is a list method only.

The classic bug

```
>>> xs = [3, 1, 2]
>>> xs = xs.sort()
>>> print(xs)
```

None

• `sort()` returns `None`, because it has already done its work in-place.

Sorting with a key function

```
>>> L = ['wasp', 'whale', 'fox', 'elephant']  
>>> sorted(L, key=len, reverse=True)  
elephant  
whale  
wasp  
fox
```

key is a **function**, applied to each item to get the value to sort on. Again we have passed-in a function as an argument.

Collecting with ordering

What are the heaviest individual parts?

You cannot sort dicts directly — Python does not know what you mean. Tell it:

```
>>> def get_mass(p): return p["mass"]
>>> heavy = sorted(parts(bicycle), reverse=True, key=get_mass)
>>> [p["name"] for p in heavy[:3]]
['frame', 'rim', 'tyre']
>>> max(parts(bicycle), key=lambda p: p["mass"])["name"]
'frame'
```

Notice the `lambda` function we use as a key. It is an **anonymous** function, but otherwise, it is exactly the same as `get_mass`.

Part 4: One part at a time

We built a list we did not need

What is the first steel part?

```
def first_steel(node):  
    for p in parts(node):  
        if p["material"] == "steel":  
            return p
```

parts walks the entire bicycle and builds all 8 entries. We looked at two of them and threw the rest away.

yield, instead of return

```
def parts(node): # original version
    if "children" not in node:
        return [node]
    out = []
    for c in node["children"]:
        out.extend(parts(c))
    return out

def parts(node): # new generator version
    if "children" not in node:
        yield node
    else:
        for c in node["children"]:
            yield from parts(c)
```

Any def containing yield is a **generator function**. return hands back a list. yield hands back one element, and the function **pauses**, to be resumed later.

yield from

`yield from` parts(c)

is the short way to write

```
for p in parts(c):  
    yield p
```

“Yield each of the things that parts(c) yields.”

Nothing happens until you ask

Calling a generator function does **not** run the body.

```
>>> ps = parts(bicycle)
>>> ps
<generator object parts at 0x1033a98c0>
>>> next(ps) ["name"]
'rim'
>>> next(ps) ["name"]
'spoke'
```

Each `next` runs the function until the next `yield`, and then stops.

But usually we iterate over a generator using a `for` loop. It calls `next` internally.

Laziness

Generators are **lazy**: they don't compute more than is asked for. Let's prove this:

```
def first_steel(node):  
    for p in parts(node):  
        print(f"visiting {p['name']}")  
        if p["material"] == "steel":  
            print(f"found {p['name']}")  
            return p
```

...

visiting rim

visiting spoke

found spoke

It **stops** after finding the item we wanted, without creating unnecessary items.

What a generator can't do

```
>>> len(parts(bicycle))
```

```
TypeError: object of type 'generator' has no len()
```

```
>>> parts(bicycle)[0]
```

```
TypeError: 'generator' object is not subscriptable
```

A generator can't do length and indexing: it does not know what it has not produced yet. But it's easy to convert generator output to a list:

```
>>> len(list(parts(bicycle)))
```

```
8
```

A generator gets used up

```
>>> ps = parts(bicycle)
>>> len(list(ps))
8
>>> len(list(ps))
0
```

It is a **one-shot** stream, not a container. The second `list` finds it already exhausted. Call `parts(bicycle)` again to get a fresh one.

This is a common bug.

Everything else still works

Counter, sorted, max and a for loop all take any iterable, so they don't care whether the input is a list or a generator.

```
>>> Counter(p["material"] for p in parts(bicycle))  
Counter({'steel': 4, 'aluminium': 2, 'rubber': 1, 'plastic': 1})
```

And in a doctest, wrap the call in `list` so there is something to compare:

```
>>> [p["name"] for p in parts(bicycle)][:3]  
['rim', 'spoke', 'hub']
```

List or generator?

	list	generator
when the work happens	all at once	one item at a time
memory	all n items	one item
stop early	time wasted	nothing wasted
len, indexing	yes	no
iterate multiple times	yes	no

Which to use? Use a generator when the sequence is long or you will not need all of it.

Part 5: data instead of code

The bill of materials should be data

Right now the bicycle is written in our program. For a new bicycle we would have to edit the code. Instead it should be **data**, not code.

Reminder: file I/O with plain text files

```
with open("example.txt") as f:  
    text = f.read() # reading  
  
with open("example_output.txt", "w") as f:  
    f.write("some example text\nanother line") # writing
```

Notice the latter will overwrite an existing file of that name, if it exists.

The bill of materials should be data

But we have structured data, not plain-text data. We will use **JSON** format:

```
{"name": "bicycle", "qty": 1, "children": [  
    {"name": "frame", "qty": 1, "material": "steel", "mass": 2200,  
    "cost": 95.0}  
]}
```

JSON looks quite like Python: strings, numbers, lists, dicts. But there are some small differences.

Reading JSON

```
with open("bicycle.json") as f:  
    bom = json.load(f)
```

- `json.load(f)` reads a JSON file into dicts and lists.
- `json.dump(obj, f)` writes to a JSON file on disk.

File locations (not examinable)

```
import json
from pathlib import Path
```

```
DATA = Path(__file__).parent / "data"
```

```
def load_bom(filename):
    with open(DATA / filename) as f:
        return json.load(f)
```

- `Path(__file__).parent` is the folder **this source file** lives in, so the program works no matter which directory you run it from.
- `/` joins paths using the OS-specific separator.

The program no longer contains a bicycle

```
>>> bom = load_bom("bicycle.json")
>>> unit_mass(bom, keep=is_steel)           # as before
3220
```

Only the source of the data moved, out of the program and into a file.

The file gives each part a cost as well as a mass. In the exercises we will sum the cost instead of the mass.

Debugging recursion

You cannot easily see recursion in a traceback. One solution is print, and indent by depth:

```
def unit_mass(node, depth=0):
    print("  " * depth + f"enter {node['name']}")
    if "mass" in node:
        result = node["mass"]
    else:
        result = sum(c["qty"] * unit_mass(c, depth + 1)
                     for c in node["children"])
    print("  " * depth + f"leave {node['name']} = {result}")
    return result
```

What it shows you

```
enter wheel
  enter rim
  leave rim = 380
  enter spoke
  leave spoke = 5
  enter hub
  leave hub = 200
  enter tyre
  leave tyre = 320
leave wheel = 1060
```

The indentation shows the call stack. Every enter has a matching leave at the same depth, and you can see each answer being built from the ones below it.

Every leave is a mass of *one*. The 32 is nowhere in the trace, because the wheel applied it: $380 + 32 \times 5 + 200 + 320$.

depth + 1 is passed down; nothing is stored. Add the depth argument while debugging, take it out afterwards.

Summary

- A bill of materials is a **tree**, so we chose a representation before writing any code: nested dicts, where a part has children or a mass.
- **Recursion**: the shape of the code matches the shape of the data. One base case, one recursive case, and the quantity applied at the right level.
- The same walk answers every question: total mass, count by type, all the parts.
- **Generators** hand back one part at a time, so we can stop early.
- A **Comprehension** is like an inside-out for loop.
- The bicycle became **data in a file**, so our code is generic and reusable.

New this week: nested data structures; list and dict **comprehensions**; sum, len, max; sort versus sorted and the key= function; conditional expressions; anonymous lambda functions; **generators** and generator comprehensions, yield, yield from, next, list(); json.load/json.dump.